LECTURER: MAJA POPOVIĆ

REINFORCEMENT LEARNING



Who am I?

- Maja Popović
- Academic Lecturer at IU since December 2023

  Previously:

  Dublin City University, Ireland (still there, half-time)

  DFKI -- German Centre for Artificial Intelligence

  RWTH Aachen

- over 15 years of experience in NLP (mainly Machine Translation)

REINFORCEMENT LEARNING
TOPIC OUTLINE

Introduction to Reinforcement Learning	1
Dynamic Programming and Markov Decision Processes	2
Bandit Problems	3
Q-Learning	4
Reinforcement Learning Approaches	5
Summary: Reinforcement Learning	6

UNIT 1

INTRODUCTION TO REINFORCEMENT LEARNING

STUDY GOALS



- Understand **what is reinforcement learning** and identify the various problems addressed by it

- Compare and contrast **how reinforcement learning surpasses challenges** encountered by traditional control theory methods

- Categorize the different **approaches** employed **in reinforcement learning**

EXPLAIN SIMPLY



1. Explain the relationship between reinforcement learning and dynamic programming to address control problems

2. Describe the key components of reinforcement learning systems, including policy evaluation and iterative updating

3. Compare on-policy and off-policy reinforcement learning methods

- What is reinforcement learning?
- Important concepts:

1) observation, action, reward
2) agent, environment
3) sequential decision process, Markov decision process, history, state
4) policy, value function, model
5) planning and learning, dynamic programming
6) exploration and exploitation
7) prediction and control
8) bellman equation, optimal value, optimal policy

Recommended sources:

- https://www.davidsilver.uk/teaching/

  on-line lectures (videos + slides)

- https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf

  on-line book

What is reinforcement learning?

WHY REINFORCEMENT LEARNING



Source of the text: James, Witten, Hastie, & Tibshirani, 2013.

Machine Learning: teaching computers to learn and decide from data autonomously

Supervised learning

- Learn the given labels
- Train using input-ouput pairs
- Goal: classification



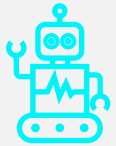
Unsupervised learning

- Find patterns in data
- No labelled data, only features
- Goal: grouping



Reinforcement learning

- Trial and error
- Agents act in environment
- Goal: learn to act



Why is reinforcement learning different?

- there is no labelled data and features, only a reward signal
- time is important
  - feedback is delayed, not instantaneous
  - sequential data, no independent identically distributed data
- agent’s actions affect the subsequent data it receives

Important concepts

1) observation, action, reward

Important concepts #1

- O_t observation at time step t (current conditions)
- A_t action at time step t
- R_t reward at time step t

2) agent, environment

Important concepts #2

- Agent
  - **performs an action**
  - **receives a reward from the environment**

- Environment
  - **gives the reward to the agent according to its action**

Agent and environment at each time step t

The agent:

- performs action A_t
- receives observation O_t
- receives scalar reward R_t

The environment:

- receives action A_t
- emits observation O_{t+1}
- emits scalar reward R_{t+1}

Reward

- a scalar feedback R_t
- indicates how well agent is doing at time step t

agent's goal = maximising cumulative reward

3) sequential decision process, Markov decision process, history, state

Important concepts #3

- Sequential decision process
- History
- Information state
- Markov decision process

Sequential decision process (SDP)

- select actions to maximise total future reward
- actions may have long term consequences
- reward may be delayed
- possibly better to sacrifice immediate reward to gain more long-term reward

History

sequence of observations, actions, and rewards:

$$H_t = O_1 A_1 R_1 O_2 A_2 R_2 \ldots O_{t-1} A_{t-1} R_{t-1} O_t A_t R_t$$

everything that happened (was observed) up to the time step t

what happens next depends on the history

Information state S_t

- contains all useful information from the history

Information state

what happens next depends on the history – how?

- state S_t = information from the history used to decide what happens next
- a function of the history: $S_t = f(H_t)$

Markov state

- only the current state is relevant for future states
- the rest of the history can be ignored

State in general:

$$P(S_{t+1}|H_t) = P(S_{t+1}|S_t, S_{t-1}, S_{t-2}, \dots S_2, S_1)$$

Markov state:

$$P(S_{t+1}|H_t) = P(S_{t+1}|S_t, S_{t-1}, S_{t-2}, \dots S_2, S_1) = P(S_{t+1}|S_t)$$

Environment state S_t^E

- environment's internal/private representation of the current situation
  - whatever the environment uses to decide on the next observation O_{t+1} and reward R_{t+1}

- a Markov state

Agent state S_t^A

- agent's internal representation of the current situation
  - whatever the agent uses to decide on the next action A_{t+1}
- the information used in RL algorithms
- a function of history $S_t^A = f(H_t)$

Markov decision process (MDP)

if the environment is fully observable
(agent directly observes the environmental state)

$$O_t = S_t^A = S_t^E$$

Almost all reinforcement learning problems can be formalised as MDP

partially observable Markov decision process

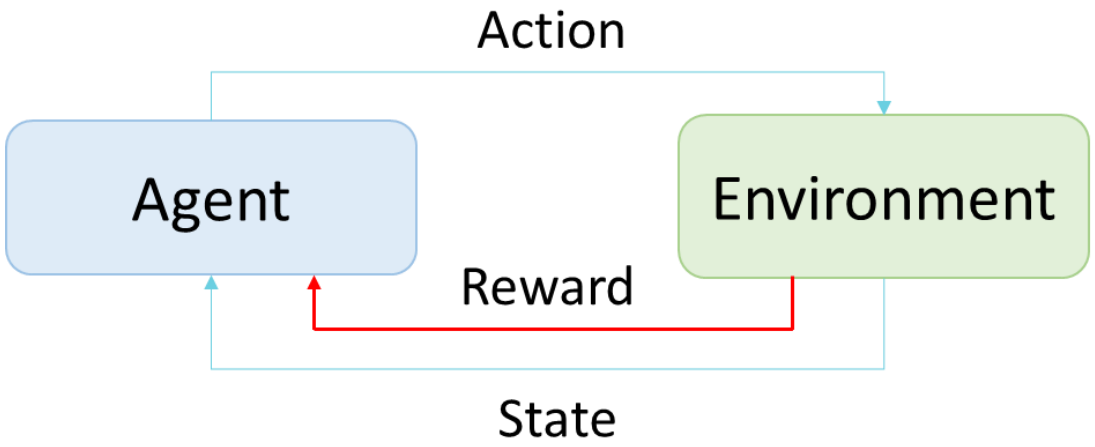
if the environment is partially observable
(agent indirectly observes the environmental state)

$$S_t^A \neq S_t^E$$

Agent must construct its state somehow, e.g.

- from the entire history $S_t^A = H_t$
- from “beliefs” about environment state $P(S_t^E = s_1), P(S_t^E = s_2), \ldots, P(S_t^E = s_n)$

REINFORCEMENT LEARNING



- **Agent**: the learning entity
- **Environment**: external context
- **State:** current condition
- **Action:** allowed move in each state
- **Reward:** incentive for action

Source of the text: Sutton & Barto, 2018.
Source of the image: Own representation.

4) policy, value function, model

Important concepts #4: components of an agent

- Policy
  agent's action as a function of state
- Value function
  how good is each action/state
- Model
  agent's representation of the environment

Policy

mapping from state to action

- deterministic policy:

$$a = \pi(s)$$

- stohastic policy:

$$\pi(a|s) = P(A_t = a \mid S_t = s)$$

Value function

prediction of future reward

used to evaluate goodness/badness of states
⇒ to select the action

$$v_\pi(s) = E_\pi[R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \cdots \mid S_t = s]$$

expected total future reward (cumulative reward, return) for the current state

Value function: cumulative reward

cumulative reward, return

$G_t = R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \cdots$

$\beta \in [0,1]$ -- discount factor

Value function: cumulative reward -- discount

Why discount?

- mathematically convenient
- avoids infinite loops in cyclic Markov processes
- uncertainty about the future
- animal/human behaviour shows preferences for immediate reward over delayed one

sometimes possible to use undiscounted reward ($\beta = 1$)
(e.g. if all sequences terminate)

Value function: state and action

state value function:

$v_\pi(s) = E_\pi[G_t \mid S_t = s]$

expected return starting from the state s and then following the policy π

action value function:

$q_\pi(s, a) = E_\pi[G_t \mid S_t = s, A_t = a]$

expected return starting from the state s, taking the action a, and then following the policy π

Model

predicts the environment's behaviour

- next state:

$$P^{a}_{s's} = P(S_{t+1} = s' \mid S_t = s, A_t = a)$$

- next (immediate) reward:

$$R^{a}_{s} = E[\, R_{t+1} \mid S_t = s, A_t = a]$$

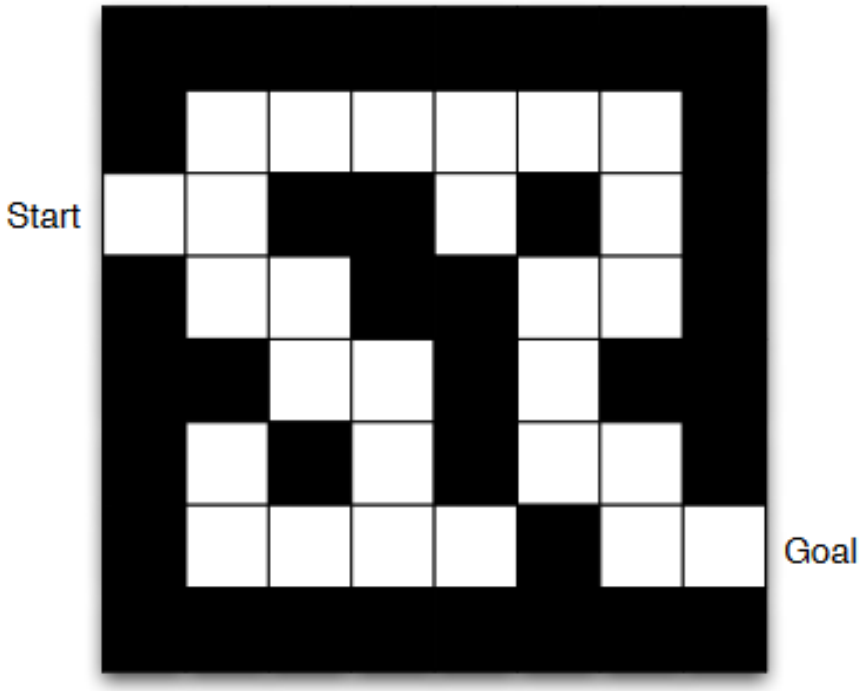
Example: maze

- States:

  agent's position in the maze
- Actions:

  moving up, down, left, right
- Reward:

  decreasing distance to the goal

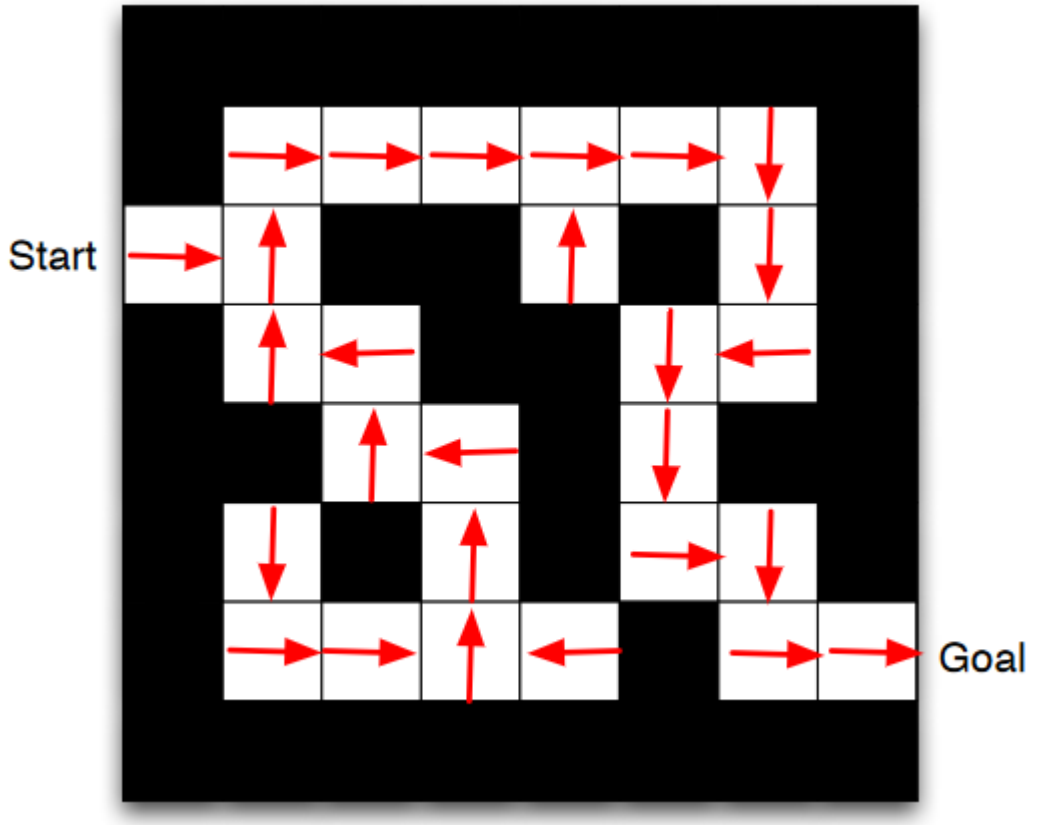


Example: maze

Policy

arrows = $\pi(s)$

selected action in each state s

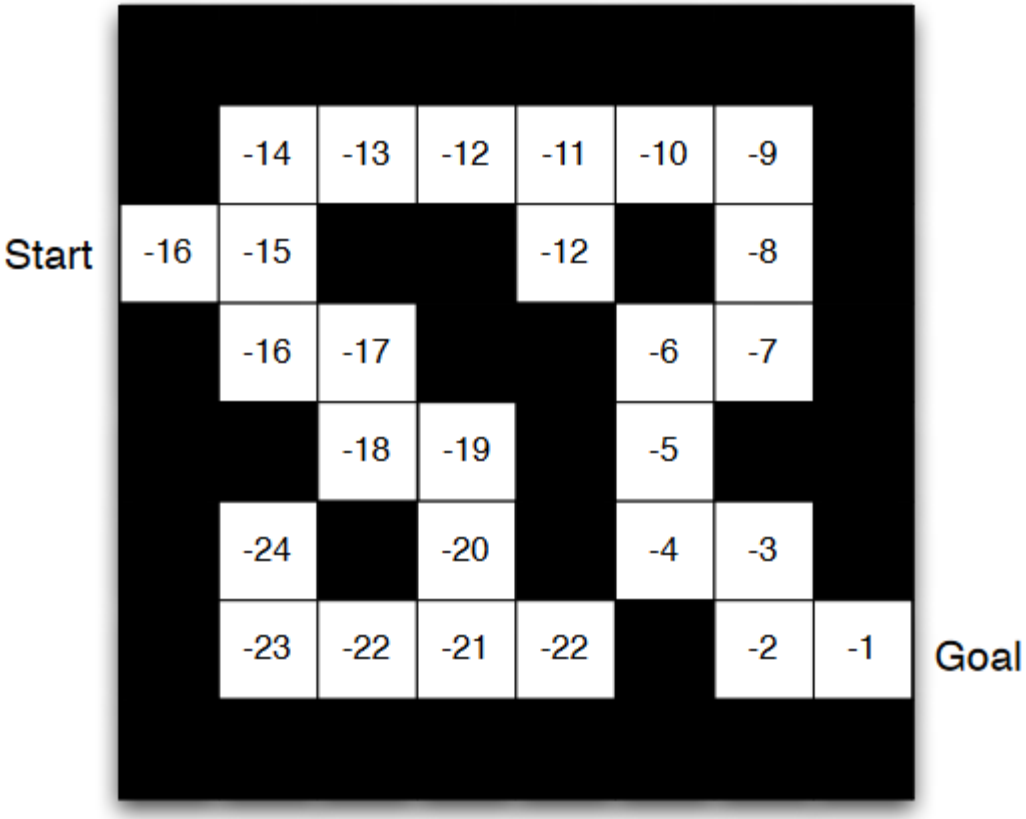


Example: maze

Value function

numbers = $v_\pi(s)$

total distance from each state s to the goal

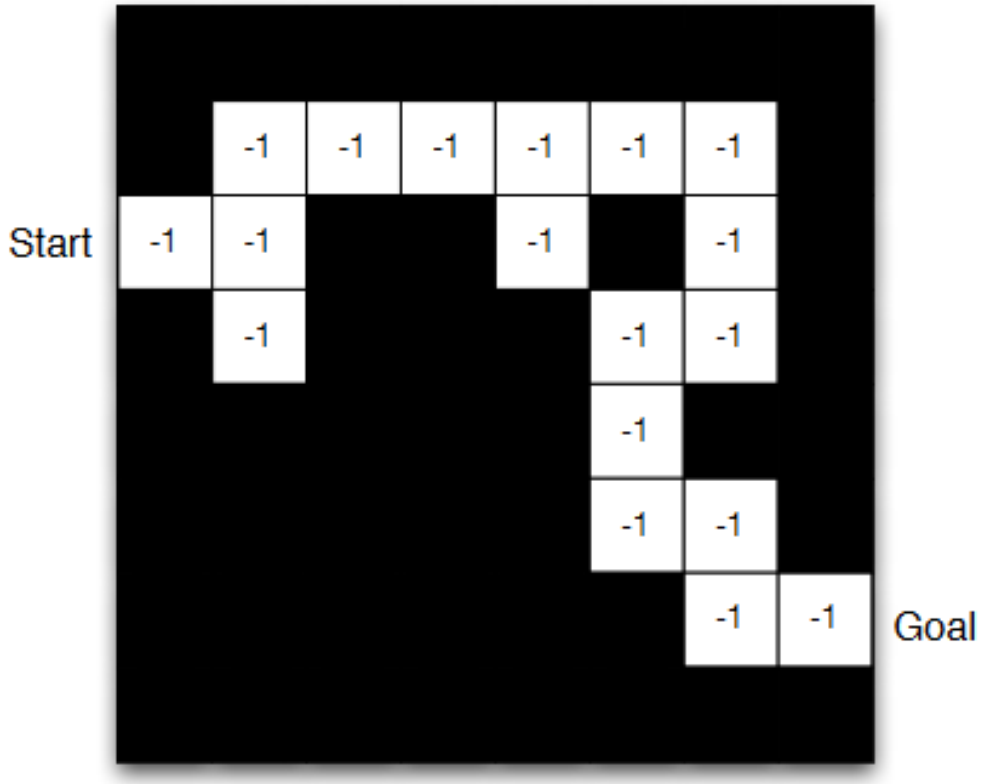


Example: maze

Model

possible states according to the model $P^a_{s's}$

numbers $= R^a_s$
(immediate reward in each state s)



Types of RL agents

Value-based	Policy-based	Actor critic
no policy	policy	policy
value function	*no value function*	value function

Model-based	Model-free
policy and/or value function	policy and/or value function
model	*no model*

5) planing and learning, dynamic programming

Important concepts #5

- Planning and learning
- Dynamic programming (for planning)

Two types of sequential decision processes:

- Planning
  - **environment model is known**
  - **the agent does not interact with the environment**
  - **the agent uses the environment's model**
  - **the agent improves its policy**

- Reinforcement learning
  - **the environment model is unknown**
  - **the agent interacts with the environment and learns from these interactions**
  - **the agent improves its policy**

Dynamic programming

- A method for solving complex problems
  by breaking them into sub-problems
    solve sub-problems and combine the solutions

- suitable for solving MDPs

Dynamic programming

Used for many other problems

- string algorithms (e.g. sequence alignment)
- graph algorithms (e.g. shortest path algorithms)
- bioinformatics (e.g. protein folding)
- …

Dynamic programming

- requires full knowledge about MDP

⇒ suitable for *planning*

6) exploitation, exploration

Important concepts #6

- exploitation
- exploration

Exploitation and exploration

Reinforcement learning -- trial-and-error learning

agent should discover a good policy

- from its interactions with the environment
- without losing too much reward along the way

Exploration

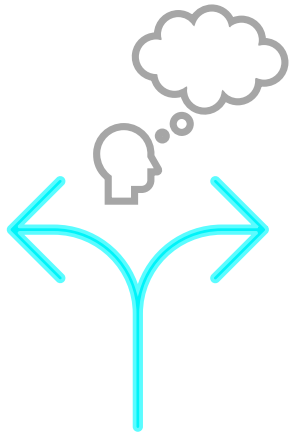
— find as much as possible information about the environment

Exploitation

— exploit known information to maximise the reward

both are important

EXPLORATION VS EXPLOITATION DILLEMA



Exploitation

- Choose high reward actions
- Exploit learned information
- Maximize immediate gain

Exploration

- Discover novel actions
- Learn about environment
- Balance knowledge expansion

When to explore new options and when to exploit what we know?

7) prediction, control

Important concepts #7

- Prediction
  - **evaluate the future reward for a given policy**
    policy evaluation

- Control
  - **optimise the future reward by finding the best policy**
    policy improvement

8) Bellman equation, optimal value function, optimal policy

Important concepts #8

- Bellman equation
- optimal value function
- optimal policy

Bellman equation

value function:

$$v_\pi(s) = E_\pi[R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \cdots \mid S_t = s]$$

$$v_\pi(s) = E_\pi[G_t \mid S_t = s]$$

cumulative reward/return:

$$G_t = R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \cdots$$

total future reward from time step t

Bellman equation

$$
\begin{aligned}
v_\pi(s) &= E_\pi[G_t \mid S_t = s] \\
&= E_\pi[R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \beta^3 R_{t+4} + \ \ldots \mid S_t = s] \\
&= E_\pi[R_{t+1} + \beta(R_{t+2} + \beta R_{t+3} + \beta^2 R_{t+4}) + \ \ldots) \mid S_t = s] \\
&= E_\pi[R_{t+1} + \beta G_{t+1} \mid S_t = s] \\
&= E_\pi[R_{t+1} + \beta v_\pi(S_{t+1}) \mid S_t = s]
\end{aligned}
$$

Bellman equation

$$v_\pi(s) = E_\pi[R_{t+1} + \beta v_\pi(S_{t+1}) \mid S_t = s]$$

value function decomposed into two components:

- immediate reward
- (discounted value) function of successor state

decomposed into “sub-problems” – dynamic programming

Bellman equation

The decomposition holds both for state value function:

$$v_\pi(s) = E_\pi[R_{t+1} + \beta v_\pi(S_{t+1}) \mid S_t = s]$$

as well as for action value function:

$$q_\pi(s, a) = E_\pi[R_{t+1} + \beta q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

Optimal value function

$$v_*(s) = \max_\pi v_\pi(s)$$

$$q_*(s,a) = \max_\pi q_\pi(s,a)$$

the maximal value function over all policies π

- specifies the best performance in a Markov decision process
  - a MDP is “solved” if the optimal value function is known

Optimal policy

A policy π is better than a policy π’

$$\pi > \pi'$$

if its value is higher for each state:

$$\forall s\colon\ v_{\pi}(s) \geq v_{\pi'}(s)$$

Optimal policy

For any Markov decision process:

exists an optimal policy π_* which is better than all other policies

$$\forall \pi: \ \pi_* \geq \pi$$

Optimal policy

Furthermore:

- All optimal policies achieve the optimal value function

$$v_{\pi*}(s) = v_*(s)$$

- All optimal policies achieve the optimal action-value function

$$q_{\pi*}(s, a) = q_*(s, a)$$

Solving Bellman optimality equation

no closed-form solution

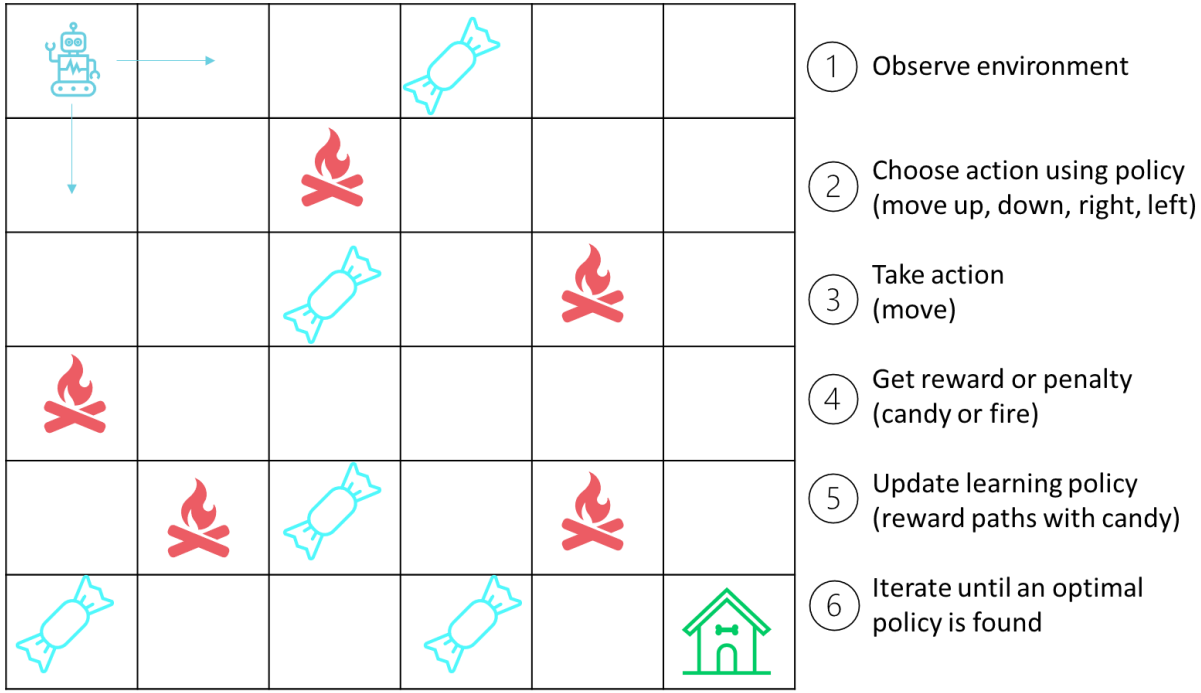
many iterative solution methods:

- value iteration
- policy iteration
- Q-learning
- SARSA

Questions?

Markov Decision Process (MDP)

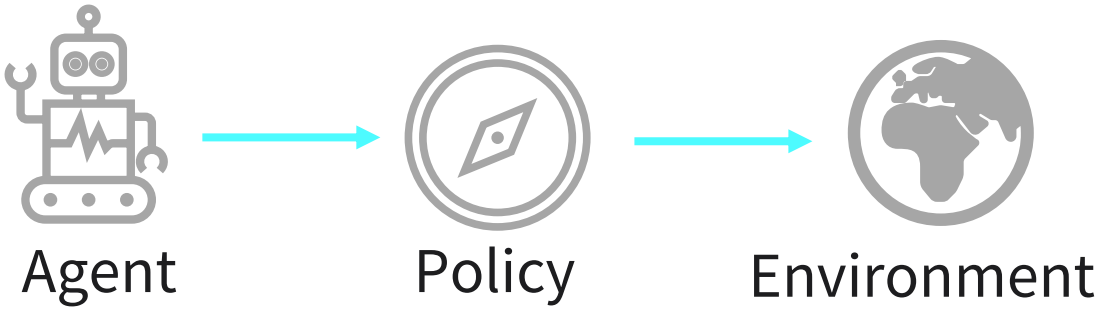
Framework to model sequential decision-making under uncertainty



1. Observe environment
2. Choose action using policy (move up, down, right, left)
3. Take action (move)
4. Get reward or penalty (candy or fire)
5. Update learning policy (reward paths with candy)
6. Iterate until an optimal policy is found

Source of the image: Own representation.

COMPONENTS OF REINFORCEMENT LEARNING



$$V^*(x) = \min_{a \in A_x} R(x,a) + \beta \sum_{y \in X} V^*(y) P(y \mid x, a)$$

- Policy maps states to actions
- Agent learns policies by experience
- optimal policy = highest reward
- **Bellman equation:** guides policy improvement and optimization
- Evaluates state-action pairs
- Cornerstone of reinforcement learning

Source of the text: Sutton & Barto, 2018.

FUNDAMENTAL REINFORCEMENT LEARNING CATEGORIES AND PRINCIPLES

Approaches

Model free

- Learn from interactions; no explicit model
- Q-learning for action-value estimation

Model based

- Use explicit model for planning
- Monte Carlo Tree Search (MCTS)

VS

On policy

- Improve using own policy's data
- SARSA for online learning

Off policy

- Learn from different policy's data
- Q-learning for off-policy learning

Categories

Source of the text: Sutton & Barto, 2018.

POLICY EVALUATION AND POLICY IMPROVEMENT



Refine policy through value assessments and action enhancements

Policy evaluation

- Initialize value of state V(s)
- Update V(s) using Bellman equation
- Repeat until convergence

Policy improvement

- Get best estimate for each state
- Look ahead to find best policy
- Pick action with highest reward

Source of the text: Sutton & Barto, 2018.



Temporal Difference Learning

Learn by comparing current and predicted outcomes

TD – Lambda

Initialize
Value estimates and eligibility traces

Observe
State, take action, receive reward.

Calculate
TD error

Update
Use TD error and eligibility trace

Repeat iteratively, adjusting trace and step size

Optimize an agent's actions by learning from experiences

Q- Learning (off-policy)

- **Explore**: Collect experience using exploration
- **Update**: Update Q-values using Bellman equation
- **Exploit**: Choose actions based on updated Q-values.
- **Iterate**: Repeatedly learn and adjust Q-values.

SARSA (on-policy)

- **Explore**: Epsilon-greedy action selection
- **Act**: Perform chosen action
- **Receive**: Get reward, observe new state
- **Update**: Adjust Q-value based on observed outcome.

REVIEW STUDY GOALS



- Understand **what is reinforcement learning** and identify the various problems addressed by it

- Compare and contrast **how reinforcement learning surpasses challenges** encountered by traditional control theory methods

- Categorize the different **approaches** employed **in reinforcement learning**

SESSION 1

TRANSFER TASK



Case study



An e-commerce giant wants to optimize its warehouse operations using autonomous robots. The robots need to navigate through the warehouse, locate items, avoid obstacles, and efficiently transport packages to packing stations

Task

Explain how this situation can be translated into a reinforcement learning problem. Define the key components required for modeling this task

TRANSFER TASK
PRESENTATION OF THE RESULTS

Please present your results.

The results will be discussed in plenary.





1. A reinforcement learning application is designed to play a card game automatically. A play holds five cards from a standard deck of 52. At each stage the player can discard and replace zero, one or two cards. How many actions are available at each stage
   a) 2
   b) 10
   c) 16
   d) 26



2. Which statement best describes the exploration-exploitation dilemma?
   a) This occurs when the decision maker wishes to both apply the best available policy and collect new data in order to improve the policy
   b) This arises because determination of the optimal policy requires statistical estimation
   c) If the environment changes, the current optimal policy is no longer optimal
   d) In general, a reinforcement learning application must visit every state-action pair infinitely often in order to converge to the optimal policy



3. If for a minimum cost MDP the discount rate is $\beta = 0.0$, how would an optimal policy be calculated
   a) The total discounted cost would always be zero, so any policy would be optimal
   b) From any state x the optimal action a would be the one minimizing R(x,a), where x is held fixed
   c) By using value iteration after setting R(x,a) to zero
   d) The problem is not well-defined, so no optimal policy exists

LIST OF SOURCES

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning* (Vol. 112, p. 18). New York: Springer.

Sutton, R. S., & Barto, A. G. (2018a). Reinforcement learning: An introduction (2nd ed.). MIT press.

© 2022 IU Internationale Hochschule GmbH
This content is protected by copyright. All rights reserved.
This content may not be reproduced and/or electronically edited, duplicated, or distributed in any kind of form without written permission by the IU Internationale Hochschule GmbH.